RugRisk

Install & Run Guide for Windows, macOS and Linux

Version 1.0 | November 2026 | Self-hosted Solana memecoin screener and rug-risk scanner

At a glance	
What you will do	Create a free database, install Python and a code editor, edit two settings files, and start RugRisk on your own computer.
Time needed	About 30 to 45 minutes the first time. Starting it again later takes one command.
Cost	Free. Everything runs on free tiers (Supabase, Helius, Jupiter) and your own computer.
Skills needed	None. You will copy and paste commands and edit two text files in VS Code or VSCodium. No nano or vim.
What you end up with	A website at http://localhost:8000 with a live token screener (filters for age, market cap, liquidity and volume), a watchlist of tokens you star, and a page where you can paste any Solana token address and get a risk report.

How to use this guide

- Work through the parts **in order**. Each part ends with something you can check before moving on.
- Gray boxes contain commands. Type or paste **one line at a time** and press **Enter**. Do not type the gray box itself, only the text inside it.
- Colored bars mark which operating system a step is for: **Windows**, **macOS** or **Linux**. Follow only the bar that matches your computer and skip the others.
- Orange and red boxes are warnings. Please read them.
- RugRisk has **no login and no accounts**. It is a personal tool that runs only on your own computer.
- A **terminal** is the text window where you type commands. You will open it from inside your editor, so you never need to find it yourself.

The whole process in one minute

1. Create a free **Supabase** project (your database) and note three values.
2. Install **Python**, **VS Code or VSCodium**, and optionally **Git**.
3. Download the project and open its folder in your editor.
4. Paste two SQL files into Supabase to create the database tables.
5. Copy two example files, fill in your values, and save.
6. Install the app's dependencies with two commands.
7. Run the self-check, then start RugRisk and open it in your browser.

Keep your secret key secret

RugRisk uses two kinds of Supabase key. The **publishable** key may be used in the website. The **secret** key must **never** be shared, pasted into chat, put in the website files, or uploaded to GitHub. Anyone with your secret key can read and change your whole database.

If a secret key is ever exposed, open Supabase, go to **Settings, API Keys**, create a new secret key and delete the old one.

Contents

- Part 1. Create your free accounts and collect your keys
- Part 2. Install the tools (Python, editor, Git)
- Part 3. Get the project and open it in your editor
- Part 4. Set up the database
- Part 5. Configure the app (edit two files)
- Part 6. Install the app's dependencies
- Part 7. Test your setup
- Part 8. Run RugRisk
- Part 9. Everyday use
- Part 10. Troubleshooting
- Appendix A. Settings reference | Appendix B. What is in the folder | Appendix C. Reset and uninstall

Part 1. Create your free accounts and collect your keys

Everything in this part happens in your web browser and is the same on every operating system. You will end up with up to five values. Keep a notes file open to paste them into as you go.

Step 1.1 Create a Supabase project (your database)

1. Go to **supabase.com** and click **Start your project**. Sign up with GitHub or with your email address.
2. Click **New project**. If asked for an organization, keep the default one.
3. Set **Name** to `rugrisk`.
4. Set a **Database password**. Use the **Generate** button and save the password in your notes. You will rarely need it, but you cannot see it again.
5. Choose the **Region** closest to you.
6. Make sure the plan is **Free**, then click **Create new project**.
7. Wait one to two minutes until the project dashboard finishes loading.

> **NOTE**
> Free Supabase projects pause after a week with no activity. RugRisk's workers write to the database regularly, so this only matters if you stop using it for more than a week. A paused project can be restored with one click from the dashboard.

Step 1.2 Find your three Supabase values

1. At the top of the project dashboard click the **Connect** button. You can also open **Settings** (gear icon) and then **API Keys**.
2. Copy the **Project URL**. It looks like `https://abcdefghijkl.supabase.co`. Paste it into your notes as **Project URL**.
3. Find the **Publishable key** (starts with `sb_publishable_`). Paste it into your notes as **Publishable key**.
4. Find the **Secret key** (starts with `sb_secret_`). If there is none, open **Secret keys** and click **New secret key**. Paste it into your notes as **Secret key**.

> **TIP**
> Older projects may only show two keys called **anon** and **service_role** (the Legacy tab). They work the same way: **anon** is the publishable key and **service_role** is the secret key.

Step 1.3 Recommended: get a free Helius key

Helius provides the fast Solana connection RugRisk uses. Without it RugRisk falls back to the public Solana connection, which is slow and often rate limited, and the metadata check cannot run.

1. Go to **helius.dev** and create a free account.
2. In the dashboard open **API Keys** and copy your key. Paste it into your notes as **Helius key**.

Step 1.4 Recommended: get a free Jupiter key

The sell simulation (the honeypot check) asks Jupiter to build a test sell. It works without a key but is rate limited, so many results would show as unverified.

1. Go to **portal.jup.ag** and sign in.

2. Create an API key on the free tier and paste it into your notes as **Jupiter key**.

> **NOTE**
> Website layouts change from time to time. If a button has a slightly different name, look for the closest match.

Value	Looks like	Required?	Goes into
Project URL	`https://abcd.supabase.co`	Yes	backend/.env **and** frontend/js/config.js
Publishable key	`sb_publishable_...`	Yes	frontend/js/config.js **only**
Secret key	`sb_secret_...`	Yes	backend/.env **only**
Helius key	a long code	Recommended	backend/.env
Jupiter key	a long code	Recommended	backend/.env

Part 2. Install the tools

You need Python (runs the app), a code editor (to edit two files) and, optionally, Git. Install them using the section for your operating system, then run the checks at the end of this part.

Step 2.1 Install Python 3.10 or newer

Windows | PowerShell

Easiest way, using the built-in installer tool. Press the **Windows key**, type **PowerShell**, and open **Windows PowerShell**. Then run:

```
winget install --id Python.Python.3.12 -e
```

If `winget` is not available, download the installer from **python.org/downloads**, run it, and **tick the box "Add python.exe to PATH" on the first screen** before clicking **Install Now**.

> **WARNING**
> Close PowerShell and open a new one after installing, otherwise it will not find Python.

macOS | Terminal

macOS ships an old Python that is too old for RugRisk, so install a current one. Option A is the installer from **python.org/downloads** (download the macOS installer and double-click it). Option B uses Homebrew. Open **Terminal** (press Cmd+Space, type Terminal) and run:

Homebrew (skip the first line if you already have it)

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
brew install python
```

Linux | Terminal

Open a terminal (Ctrl+Alt+T on most systems) and run the line for your distribution:

Debian, Ubuntu, Kali, Linux Mint

```
sudo apt update && sudo apt install -y python3 python3-venv python3-pip unzip
```

Fedora

```
sudo dnf install -y python3 unzip
```

Arch, Manjaro

```
sudo pacman -S --needed python unzip
```

> **WARNING**
> On Debian and Ubuntu the package **python3-venv** is required. Without it the next steps fail with an "ensurepip is not available" error. Very old releases (for example Debian 11 or Ubuntu 20.04) ship Python older than 3.10. If your version in Step 2.4 is lower than 3.10, upgrade your system or install a newer Python.

Step 2.2 Install a code editor (VS Code or VSCodium)

Either editor works the same way for this guide. **VS Code** is from Microsoft. **VSCodium** is the community build without Microsoft's telemetry. Pick one.

Windows

VS Code

```
winget install --id Microsoft.VisualStudioCode -e
```

or VSCodium

```
winget install --id VSCodium.VSCodium -e
```

macOS

VS Code

```
brew install --cask visual-studio-code
```

or VSCodium

```
brew install --cask vscodium
```

Without Homebrew, download the app from **code.visualstudio.com** or **vscodium.com** and drag it into your Applications folder.

Linux

VS Code (snap)

```
sudo snap install code --classic
```

or VSCodium (snap)

```
sudo snap install codium --classic
```

No snap? Download the **.deb** (Debian, Ubuntu, Kali) or **.rpm** (Fedora) file from **code.visualstudio.com** or **vscodium.com**. On Debian-based systems install a downloaded file with `sudo apt install ./the-file-name.deb`. VSCodium is also available on Flathub: `flatpak install flathub com.vscodium.codium`.

Step 2.3 Install Git (optional)

Git is only needed if you want to download RugRisk from GitHub or publish your own copy. If you received a zip file, you can skip this step and install Git later.

Windows

```
winget install --id Git.Git -e
```

macOS

```
brew install git
```

Or simply type `git --version` in Terminal. macOS offers to install the developer tools, which include Git.

Linux

Debian, Ubuntu, Kali

```
sudo apt install -y git
```

Fedora

```
sudo dnf install -y git
```

Arch

```
sudo pacman -S --needed git
```

Step 2.4 Check that everything is installed

Open a **new** terminal window (important after installing) and run these. Each should print a version number.

Windows

```
python --version
git --version
```

macOS

```
python3 --version
git --version
```

Linux

```
python3 --version
git --version
```

NOTE

The Python version must be **3.10 or higher** (for example `Python 3.12.4`). Git is optional.

Windows only: if `python` shows "Python was not found; run without arguments to install from the Microsoft Store", open **Settings, Apps, Advanced app settings, App execution aliases** and switch off the two **python** entries. Or try `py --version` instead.

Part 3. Get the project and open it in your editor

Step 3.1 Get the project files

You can use the **zip file** (simplest) or **clone from GitHub** (if the project has been published there).

Windows

1. Right-click `rugrisk.zip` and choose **Extract All...**, then click **Extract**. Move the extracted folder somewhere easy, for example your Desktop.
2. Open the folder. You should see folders named **backend**, **frontend**, **scripts** and **supabase** directly inside. If you instead see only one more folder called `rugrisk`, use that inner folder.

macOS

1. Double-click `rugrisk.zip` in Downloads. A folder called `rugrisk` appears.
2. Drag it to your Documents or Desktop folder.
3. Open it and confirm you see **backend**, **frontend**, **scripts** and **supabase** inside.

Linux

```
cd ~
unzip ~/Downloads/rugrisk.zip
ls rugrisk
```

The last command should list **backend**, **frontend**, **scripts** and **supabase**.

All systems | alternative: clone from GitHub

```
git clone https://github.com/YOUR-USERNAME/rugrisk.git
```

Replace `YOUR-USERNAME/rugrisk` with the real repository address.

Step 3.2 Open the folder in your editor

1. Start **VS Code** (or **VSCodium**).
2. Click **File, Open Folder...** (on macOS: **File, Open...**).
3. Select the `rugrisk` folder, the one that directly contains backend, frontend and scripts, and confirm.
4. If a box asks "Do you trust the authors of the files in this folder?" click **Yes, I trust the authors**.

On the left you now see the **Explorer** with all the project files. Click any file to open it in the editor. Edit it, then save with **Ctrl+S** (**Cmd+S** on a Mac). A white dot on a file's tab means it is **not saved yet**.

Step 3.3 Open the built-in terminal

In the editor click **Terminal, New Terminal** (shortcut **Ctrl+`**, the backtick key). A terminal opens at the bottom, already inside the project folder. **Every command in this guide is typed here**, so you never have to navigate to folders yourself.

> **TIP**
> To check you are in the right place, type `ls` (macOS, Linux) or `dir` (Windows). You should see backend, frontend, scripts and supabase listed.

Part 4. Set up the database

RugRisk stores tokens and scan results in your Supabase project. One file of SQL (database instructions) creates everything. You will copy it from your editor and paste it into Supabase.

Step 4.1 Run the database file

1. In your editor Explorer open **supabase, migrations, 001_init.sql**.
2. Click inside the file, press **Ctrl+A** (select all) then **Ctrl+C** (copy). On a Mac use Cmd instead of Ctrl.
3. In the Supabase dashboard click **SQL Editor** in the left menu, then **New query**.
4. Click in the empty editor box and press **Ctrl+V** to paste.
5. Click **Run** (bottom right) or press **Ctrl+Enter**.
6. You should see **Success. No rows returned.** That is the correct result.

> **NOTE**
> You may also see a notice that `pg_cron` setup was skipped. That is harmless. To enable the hourly cleanup of old data, go to **Database, Extensions**, switch on **pg_cron**, and run the file again. The file is safe to run more than once.

Step 4.2 Check that the tables exist

In Supabase click **Table Editor** in the left menu. You should see three tables: **tokens**, **token_scans** and **worker_state**. They are empty for now; RugRisk fills them once it runs.

> **NOTE**
> **Upgrading from an earlier version?** Run `001_init.sql` again. It is safe to repeat, and it adds the watchlist column and the longer 8-day retention. If your earlier version also had user accounts, optionally run **supabase, optional, remove_accounts.sql** afterwards.

> **TIP**
> Used an earlier version of RugRisk that had user accounts? Running the new file on top is fine. To remove the old unused account tables, optionally run **supabase, optional, remove_accounts.sql** the same way. Skipping it is harmless.

Part 5. Configure the app (edit two files)

RugRisk reads two settings files. They are not included in the download (so secrets never end up in a shared copy), so you create each one by copying an example file, then edit it in your editor.

Step 5.1 Create the two settings files

In the terminal at the bottom of your editor, run the two lines for your system:

Windows

```
Copy-Item backend\.env.example backend\.env
Copy-Item frontend\js\config.example.js frontend\js\config.js
```

macOS

```
cp backend/.env.example backend/.env
cp frontend/js/config.example.js frontend/js/config.js
```

Linux

```
cp backend/.env.example backend/.env
cp frontend/js/config.example.js frontend/js/config.js
```

> **TIP**
> **Prefer clicking?** In the editor Explorer right-click `.env.example`, choose **Copy**, right-click the **backend** folder, choose **Paste**, then right-click the pasted copy and **Rename** it to exactly `.env`. Do the same for `config.example.js` in frontend/js, renaming the copy to `config.js`.
>
> The file must be called exactly `.env` (not `.env.txt`). Using the commands above avoids that mistake.

Step 5.2 Edit backend/.env

Open **backend, .env** in the editor. Find these four lines (the first two are at the top; the Helius and Jupiter lines are a little further down) and replace the placeholder text after the `=` sign with your own values from Part 1. Leave every other line as it is.

What the lines should look like (your values will differ)

```
SUPABASE_URL=https://abcdefghijkl.supabase.co
SUPABASE_SECRET_KEY=sb_secret_xxxxxxxxxxxxxxx
HELIUS_API_KEY=your-helius-key
JUPITER_API_KEY=your-jupiter-key
```

> **WARNING**
> In the `.env` file write values **with no quotes and no spaces** around the `=` sign.
>
> Right: `SUPABASE_URL=https://abc.supabase.co` Wrong: `SUPABASE_URL = "https://abc.supabase.co"`
>
> If you skipped the Helius or Jupiter keys, leave those two lines empty (nothing after the `=`).

Save the file with **Ctrl+S** (**Cmd+S** on a Mac). The white dot on the tab should disappear.

Step 5.3 Edit frontend/js/config.js

Open **frontend, js, config.js**. It contains two lines to change. Use the **publishable** key here, **never** the secret key.

What the file should look like

```
export const SUPABASE_URL = "https://abcdefghijkl.supabase.co";
export const SUPABASE_KEY = "sb_publishable_xxxxxxxxxxxxxxx";
export const API_URL = "http://localhost:8080";
```

> **NOTE**
> Unlike the `.env` file, here the values **do** go inside double quotes, and each line ends with a semicolon. Leave the `API_URL` line unchanged. Save with Ctrl+S (Cmd+S).
>
> This file is the **only** place the publishable key is used. The `.env` file does not need it.

Step 5.4 Double-check before continuing

- `backend/.env` exists, is saved, and contains your URL and your **secret** key.
- `frontend/js/config.js` exists, is saved, and contains your URL and the **publishable** key (not the secret one).
- Neither file contains the words REPLACE_ME, YOUR or xxxx.

Part 6. Install the app's dependencies

RugRisk uses a small set of Python libraries. You install them into a private folder called a virtual environment (`.venv`) so nothing else on your computer is touched. Run all commands in the editor's terminal, in the project folder.

Step 6.1 Create the virtual environment and install the libraries

Windows | PowerShell

```
python -m venv backend\.venv
.\backend\.venv\Scripts\python -m pip install --upgrade pip
.\backend\.venv\Scripts\python -m pip install -r backend\requirements.txt
```

macOS

```
python3 -m venv backend/.venv
backend/.venv/bin/python -m pip install --upgrade pip
backend/.venv/bin/python -m pip install -r backend/requirements.txt
```

Linux

```
python3 -m venv backend/.venv
backend/.venv/bin/python -m pip install --upgrade pip
backend/.venv/bin/python -m pip install -r backend/requirements.txt
```

The install takes a minute and prints many lines. It has worked if the last lines say `Successfully installed` and list fastapi, requests, uvicorn and others.

> **TIP**
> Notice that you never have to "activate" anything. Each command uses the virtual environment's own Python directly, which avoids the most common beginner errors (such as Windows blocking scripts, or the Linux "externally-managed-environment" error).

Part 7. Test your setup

RugRisk includes a self-check that tests your database, your keys, every outside service and a full scan, and tells you exactly what is wrong if anything is. Run it before starting the app.

Step 7.1 Run the self-check

Windows

```
.\backend\.venv\Scripts\python backend\selfcheck.py
```

macOS

```
backend/.venv/bin/python backend/selfcheck.py
```

Linux

```
backend/.venv/bin/python backend/selfcheck.py
```

It takes 20 to 60 seconds. It reads your settings, tests the database's security, calls each outside service, and scans the well-known BONK token from start to finish.

Step 7.2 Read the results

Each line begins with **PASS**, **WARN** or **FAIL**. A healthy run looks like this (your numbers will differ):

Example output

```
RugRisk self-check

[PASS] env: required variables present
[PASS] rpc config: custom RPC configured
[PASS] jupiter key: set
[PASS] supabase tables: tokens, token_scans and worker_state are readable with the secret
key
[PASS] watchlist column: tokens.watchlisted_at exists
[PASS] frontend config: config.js has a project URL and a non-secret key
[PASS] website key can read tokens: HTTP 200
[PASS] website key CANNOT write tokens: HTTP 401
[PASS] website key cannot read worker_state: HTTP 401
[PASS] website key cannot run maintenance functions: HTTP 404
[PASS] DexScreener: pairs returned
[PASS] GeckoTerminal: HTTP 200
[PASS] Jupiter quote: route found
[PASS] Solana RPC: slot 3xxxxxxxx (140 ms, helius=True)
   scan of DezXAZ8z... took 9.2s -> score 10/100 LOW ...
[PASS] BONK mint_authority is revoked

0 failed, 0 warnings
```

You see	What it means	What to do
PASS	That part works.	Nothing.
WARN	It works, but a feature will run in a reduced mode (for example no Jupiter key, so the sell simulation may show a question mark).	Optional. Add the missing key to `backend/.env` and run the check again.
FAIL env	Your `.env` is missing, misnamed, unsaved, or the URL or secret key is empty.	Redo Part 5, Step 5.2. Make sure the file is saved and named exactly `.env`.
FAIL table ... HTTP 404	The database file was not run.	Redo Part 4.

You see	What it means	What to do
FAIL watchlist column	Your database was created by an older version.	Run the latest `001_init.sql` again (Part 4, Step 4.1). It is safe to repeat.
FAIL table ... HTTP 401 / 403	Wrong secret key or URL.	Copy both again from Supabase into `.env` and save.
FAIL frontend config	`frontend/js/config.js` is missing, still has placeholders, or contains a **secret** key.	Redo Part 5, Step 5.3 with the **publishable** key. If a secret key was in that file, create a new secret key in Supabase and delete the old one.
FAIL website key CANNOT write tokens ... HTTP 200 / 201	A **security problem**: the database lockdown did not apply.	Run `001_init.sql` again (Part 4, Step 4.1), then re-run the check.
FAIL network error	No internet, a VPN or firewall is blocking, or a service is down.	Check your connection, wait a minute, run it again.

> **TIP**
> To test a specific token (for example one that just graduated from pump.fun) add `--address` and the token address at the end of the command. Example (macOS and Linux): `backend/.venv/bin/python backend/selfcheck.py --address <TOKEN_ADDRESS>`.

Part 8. Run RugRisk

One command starts everything: the worker that finds new tokens and keeps their prices fresh, the scan API (which also saves your watchlist stars) and the website. It works the same way on all three operating systems.

Step 8.1 Start it

Windows

```
python scripts\run_local.py
```

macOS

```
python3 scripts/run_local.py
```

Linux

```
python3 scripts/run_local.py
```

You will see this message, followed by log lines from the workers scrolling past:

```
RugRisk is starting...
  Website:  http://localhost:8000
  API:      http://localhost:8080/health
  Press Ctrl+C to stop everything.
```

> **NOTE**
> A firewall window may ask whether to allow Python. RugRisk only listens on your own computer (localhost), so both **Allow** and **Cancel** are safe. Never forward these ports or expose them to the internet: RugRisk has no login.
>
> If the window ends right away with a message like `'ingest' stopped unexpectedly`, scroll up: the line above it names the problem. See Part 10.

Step 8.2 Open the website

Open your web browser and go to **http://localhost:8000**. Leave the terminal running while you use the site.

Step 8.3 What you should see

After	What appears
A few seconds	The Screener page with a filter panel and a table that says Loading, then lists tokens.
1 to 2 minutes	Rows fill with new tokens, prices, liquidity and volume. The table refreshes every 30 seconds.
Hours to days	The older-age filters (24 hours, 1 week) fill up as RugRisk keeps tracking tokens. It keeps them for about 8 days; starred tokens are kept until you remove them.

Step 8.4 Filter, star and watch tokens

1. **Min age:** the dropdown (Any, 5 min, 15 min, 1 hour, 6 hours, 24 hours, 1 week) hides tokens younger than the value you pick. Choose **24 hours** to see only tokens that have survived at least a day.
2. **Market cap:** type a **Min** and/or **Max** value in dollars. For example Min `100000` and Max `5000000` shows tokens worth between $100K and $5M. Tokens without a known market cap are hidden while either box is filled.

3. Use **Min liquidity**, **Min 1h volume**, **Min buy %** and **Sort by** to narrow further, or click a quick preset such as **Survived 24h+** or **Under $1M mcap**. **Reset** clears everything. Your last filters are remembered.
4. Click the **star** at the start of any row to add the token to your **watchlist**. The star turns gold. Click it again to remove the token.
5. Open **Watchlist** at the top of the page to see all starred tokens in one table. Starred tokens are never deleted by the cleanup and their prices are refreshed every round. Click a star there to remove a token.

> **NOTE**
> Stars and filters need the local scan service to be running, because it saves your stars. If a star click shows "Couldn't update the watchlist", check that the terminal running RugRisk is still open.

Step 8.5 Scan a token for risks

1. Click the **Scan** link at the end of any row. The risk report opens and a scan starts automatically if there is no recent one (5 to 15 seconds).
2. Or click **Scan** at the top, paste any Solana token address and press **Look up**. For a token you have not scanned yet, click **Scan now**. No account or login is needed.
3. The report shows a gauge with the 0 to 100 score, every check with a plain-English reason, and a note listing anything that could not be verified. Use **Add to watchlist** on that page to star the token.

A token that was scanned in the last 3 minutes is reused instead of repeated, which saves your Solana connection quota. The button then changes to **Re-scan now**.

Step 8.6 Stop it

Click inside the terminal and press **Ctrl+C**. After a moment everything stops and you get your prompt back.

Part 9. Everyday use

Start it again later

1. Open your editor, then **File, Open Recent** and pick the `rugrisk` folder.
2. Open the terminal (**Ctrl+**`) and run the start command from Part 8.
3. Go to http://localhost:8000.

Scan one token from the command line

Windows

```
.\backend\.venv\Scripts\python backend\scan.py <TOKEN_ADDRESS>
```

macOS

```
backend/.venv/bin/python backend/scan.py <TOKEN_ADDRESS>
```

Linux

```
backend/.venv/bin/python backend/scan.py <TOKEN_ADDRESS>
```

Add `--json` for machine-readable output, or `--save` to store the result in your database so it shows up in the screener.

Update to a newer version

If you cloned with Git, run `git pull` in the project folder, then repeat the install command from Part 6 so new libraries are picked up. If you use zip files, extract the new version into a new folder and copy your `backend/.env` and `frontend/js/config.js` files across.

Make scans faster or lighter

Each full scan makes roughly 10 to 15 requests to your Solana connection. If you hit rate limits, open `backend/.env` and change one of these, save, and restart RugRisk:

- `ENABLE_HONEYPOT_SIM=0` turns off the sell simulation.
- `ENABLE_LP_CHECK=0` turns off the liquidity burn and lock check.
- `MIN_LIQUIDITY_USD=5000` stores and scans fewer low-quality tokens.

Run only parts of it

Advanced: the three processes can run separately (`ingest.py`, `uvicorn api.main:app`, and the website server). `scripts/run_local.py` is simply the convenient way to start them together.

Optional: background risk scorer

RugRisk no longer scans every new token automatically: scans run when you click **Scan**, which saves your Solana connection quota. If you want every new token scored in the background anyway (the results then load instantly on the Scan page), start RugRisk with an extra word:

Windows

```
python scripts\run_local.py --autoscan
```

macOS

```
python3 scripts/run_local.py --autoscan
```

Linux

```
python3 scripts/run_local.py --autoscan
```

Use `AUTOSCAN_DELAY_S` in `.env` to make it slower and gentler if you hit rate limits.

Part 10. Troubleshooting

Find your symptom in the left column. Most problems are a typo or an unsaved file.

Symptom	Likely cause	Fix
`python` is not recognized (Windows)	Python is not on PATH.	Reinstall Python and tick **Add python.exe to PATH**, or use `py` instead of `python`. Open a new terminal afterwards.
"Python was not found; run without arguments to install from the Microsoft Store"	Windows app aliases intercept the command.	Settings, Apps, Advanced app settings, App execution aliases: switch off both **python** entries.
`python3: command not found` (macOS)	Python is not installed.	Install it with Homebrew (`brew install python`) or from python.org.
"ensurepip is not available" or "python3-venv" (Linux)	The venv package is missing.	`sudo apt install python3-venv` and run the venv command again.
"externally-managed-environment"	You ran plain `pip` instead of the venv's Python.	Use the exact commands in Part 6: `backend/.venv/bin/python -m pip ...`.
Python version is lower than 3.10	Old Python.	Install a newer one (Part 2) and delete the `backend/.venv` folder, then redo Part 6.
"No module named requests" (or fastapi)	Libraries installed into a different Python.	Redo Part 6 and run everything through `backend/.venv`.
Self-check says variables are missing	`.env` is missing, misnamed (`.env.txt`), unsaved or in the wrong folder.	It must be `backend/.env`. Check the editor tab has no white dot and the name has no `.txt`.
Website shows **Setup needed**	`frontend/js/config.js` is missing, unsaved or still has the placeholder key.	Redo Part 5, Step 5.3 and refresh the page with Ctrl+F5 (Cmd+Shift+R on Mac).
Self-check FAIL: table ... 404	Database files not run.	Redo Part 4.
Self-check FAIL: ... 401 or 403	Wrong or revoked secret key, or a typo in the URL.	Copy the values again from Supabase, paste, save.
The screener table stays empty	The ingest worker is not running or cannot reach the internet.	Look at the terminal for red `ERROR` lines. Wait 2 minutes. Lower `MIN_LIQUIDITY_USD` in `.env`.
The 24 hours or 1 week age filter shows nothing	RugRisk has not been running long enough to track tokens that old. It only lists tokens it has watched since launch.	Leave RugRisk running (a day for 24 hours, a week for 1 week). Starred tokens are kept as long as you like.
Clicking a star shows "Couldn't update the watchlist"	The local scan service is not running, or your database is from an older version.	Check the RugRisk terminal is still open and run the latest `001_init.sql` once more (Part 4, Step 4.1).
Older tokens show outdated prices	Tokens older than a day are refreshed in rotation, a slice per round.	Star the ones you care about (they refresh every round) or raise `REFRESH_OLD_PER_LOOP` in `.env`.

Symptom	Likely cause	Fix
Many checks show a question mark	Missing Helius or Jupiter key, or rate limiting.	Add the keys and restart. Some checks (for example concentrated-liquidity pools) cannot be verified by design.
"Can't reach the local scan service"	The API is not running or port 8080 is busy.	Make sure the terminal running RugRisk is still open and shows no errors. Open http://localhost:8080/health, it should show ok.
"Too many scans in a minute"	You started more than 60 scans within a minute.	Wait a moment and try again. Change `REQUESTS_PER_MIN` in `.env` if you need more.
Port 8000 or 8080 already in use	A previous run is still open, or another program uses the port.	See the box below.

Port already in use

First press Ctrl+C in any older RugRisk terminal. If that does not help, find and stop the program using the port:

Windows

```
netstat -ano | findstr :8000
taskkill /PID <number-from-the-last-column> /F
```

macOS

```
lsof -i :8000
kill <PID-from-the-list>
```

Linux

```
ss -ltnp | grep 8000
kill <PID-shown-in-the-output>
```

> **TIP**
> Still stuck? Copy the **last 20 lines** from your terminal. **Remove any keys from them first.** Paste them to whoever is helping you, together with your operating system and the output of the self-check.

Appendix A. Settings reference

All settings live in `backend/.env`. Only the first two are required. (The publishable key is not here: it goes only into `frontend/js/config.js`.)

Setting	Default	What it does
`SUPABASE_URL`	(required)	Your Supabase project address.
`SUPABASE_SECRET_KEY`	(required)	Lets the backend write to the database. Never share it.
`HELIUS_API_KEY`	empty	Fast Solana connection and the metadata check. Strongly recommended.
`SOLANA_RPC_URL`	empty	Use another Solana RPC provider instead of Helius.
`JUPITER_API_KEY`	empty	Used by the sell simulation. Works without it at a low rate limit.
`ENABLE_LP_CHECK`	1	Set to 0 to skip the liquidity burn/lock check (saves about 5 requests per scan).
`ENABLE_HONEYPOT_SIM`	1	Set to 0 to skip the sell simulation.
`MIN_LIQUIDITY_USD`	1000	Tokens below this liquidity are ignored. Raise it to store and scan less.
`INGEST_INTERVAL_S` / `REFRESH_INTERVAL_S`	10 / 20	How often new tokens are fetched and prices refreshed (seconds).
`REFRESH_MAX_TOKENS`	150	Tokens younger than 24 hours whose prices are refreshed every round.
`REFRESH_OLD_PER_LOOP`	60	Older tokens refreshed per round, stalest first. Set 0 to stop refreshing old tokens.
`AUTOSCAN_BATCH` / `AUTOSCAN_DELAY_S`	5 / 2.0	Only if you start RugRisk with `--autoscan`: tokens scored at once and the pause between scans.
`ALLOWED_ORIGINS`	localhost:8000	Website addresses allowed to call the API. Change only if you serve the site elsewhere.
`SCAN_CACHE_TTL_S`	180	How long a finished scan is reused instead of repeated.
`MAX_CONCURRENT_SCANS`	4	How many scans may run at the same time.
`REQUESTS_PER_MIN`	60	Scan requests allowed per minute. Protects your Solana and Jupiter quota.

After changing `.env`, save it and restart RugRisk (Ctrl+C, then the start command).

Appendix B. What is in the folder

Path	Purpose
`backend/ingest.py`	Finds new Solana tokens and keeps prices fresh.
`backend/autoscan.py`	Optional background risk scorer (start with `--autoscan`).
`backend/api/`	The scan service used by the Scan page (port 8080).
`backend/scanner/`	The risk checks and scoring.
`backend/selfcheck.py`	Tests your whole setup.
`backend/scan.py`	Scans one token from the command line.
`backend/tests/`	Automated tests for developers.
`frontend/`	The website (Screener, Watchlist and Scan pages). Served on port 8000.
`supabase/migrations/`	The database file from Part 4.
`supabase/optional/`	Optional clean-up for databases from older versions.
`scripts/run_local.py`	Starts everything with one command.
`docs/`	These guides.

Port	What listens there
`8000`	The website. http://localhost:8000
`8080`	The scan API. http://localhost:8080/health

Both ports accept connections **only from your own computer**. Nobody else on your network can reach them. Keep it that way: the scan API has no login.

Appendix C. Reset and uninstall

- **Stop RugRisk:** Ctrl+C in its terminal.
- **Start from a clean install of the libraries:** delete the `backend/.venv` folder and repeat Part 6.
- **Clear all stored tokens:** in Supabase open **SQL Editor** and run `delete from public.tokens;`. Scan history is removed with it.
- **Remove RugRisk completely:** delete the `rugrisk` folder. Then, in Supabase, open **Settings, General** and choose **Delete project** if you no longer need the database.
- **Remove your keys:** if you shared your secret key anywhere, create a new one in Supabase (**Settings, API Keys**) and delete the old one.

Please remember
RugRisk shows risk **indicators**, not guarantees or financial advice. A low score can still be a scam, and checks marked with a question mark were not verified. Never invest money you cannot afford to lose.